

Sistema de Auditoria de Conformidade — NS Aplicação

Documentação Técnica Completa

Versão: 1.0

Data: Junho 2025

Autor: Sílvio Massango

Instituição: ISAF — Instituto Superior de Administração e Finanças

Curso: Informática de Gestão Financeira

Índice

1. Visão Geral do Sistema
 2. Arquitectura
 3. Base de Dados
 4. Motor de Regras
 5. Módulos da Aplicação
 6. Segurança e Autenticação
 7. Importação de Dados
 8. Exportação de Relatórios
 9. Interface Web
 10. Estrutura de Ficheiros
 11. Decisões Técnicas
 12. Limitações e Trabalho Futuro
-

1. Visão Geral do Sistema

O Sistema de Auditoria de Conformidade da NS Aplicação é uma aplicação web desenvolvida em Python com o framework Flask, concebida para automatizar a verificação da conformidade procedimental e orçamental nos processos de aquisição de bens e serviços.

Problema que resolve

A verificação manual da conformidade dos processos de aquisição apresenta três limitações críticas:

- **Cobertura limitada** — o auditor humano não consegue verificar sistematicamente todas as transacções

- **Rastreabilidade reduzida** — as decisões de auditoria não ficam registadas de forma estruturada
- **Detecção tardia** — as irregularidades são identificadas muito depois de ocorridas

O que o sistema faz

1. Recebe dados de aquisições importados de sistemas ERP via ficheiros CSV
2. Aplica automaticamente 16 regras de negócio sobre esses dados
3. Classifica cada aquisição como conforme ou não conforme
4. Regista todas as irregularidades detectadas com descrição, gravidade e rastreabilidade
5. Gera relatórios em PDF e Excel para suporte à decisão do auditor

O que o sistema NÃO faz

O sistema não é transaccional — não regista, aprova nem rejeita aquisições. É um sistema de análise post-hoc que audita dados já processados pelo ERP da organização.

2. Arquitectura

Stack tecnológico

Componente	Tecnologia	Versão	Justificação
Backend	Python + Flask	3.14 / 3.1.3	Microframework flexível, adequado para motor de regras customizado
ORM	Flask-SQLAlchemy	3.1.1	Abstracção da base de dados, queries tipadas
Migrações	Flask-Migrate + Alembic	4.1.0	Versionamento do schema da base de dados
Autenticação	Flask-Login	—	Gestão de sessões e protecção de rotas
Base de Dados	MySQL	8.0	SGBD relacional com suporte a constraints e transacções
Driver MySQL	PyMySQL	1.2.0	Driver Python puro, sem dependências nativas
Templates	Jinja2	3.1.6	Motor de templates integrado no Flask
Frontend	Bootstrap	5.3.0	Framework CSS responsivo via CDN
Ícones	Bootstrap Icons	1.10.0	Biblioteca de ícones via CDN
Gráficos	Chart.js	4.4.0	Visualização de dados via CDN

Componente	Tecnologia	Versão	Justificação
PDF	ReportLab	—	Geração de PDFs em Python
Excel	openpyxl	—	Leitura e escrita de ficheiros .xlsx

Padrão arquitectural

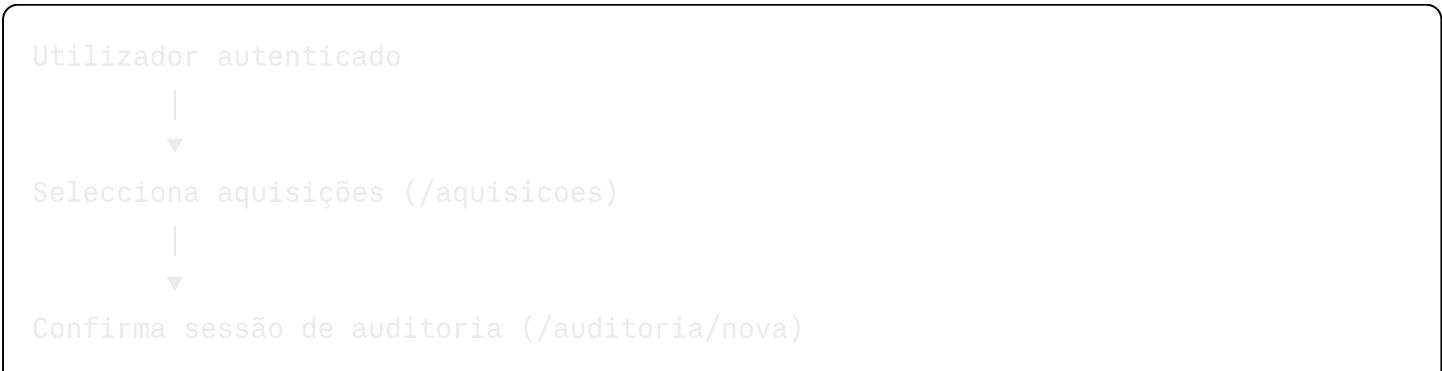
O sistema segue o padrão **Application Factory** com **Blueprints**, organizado em três camadas:

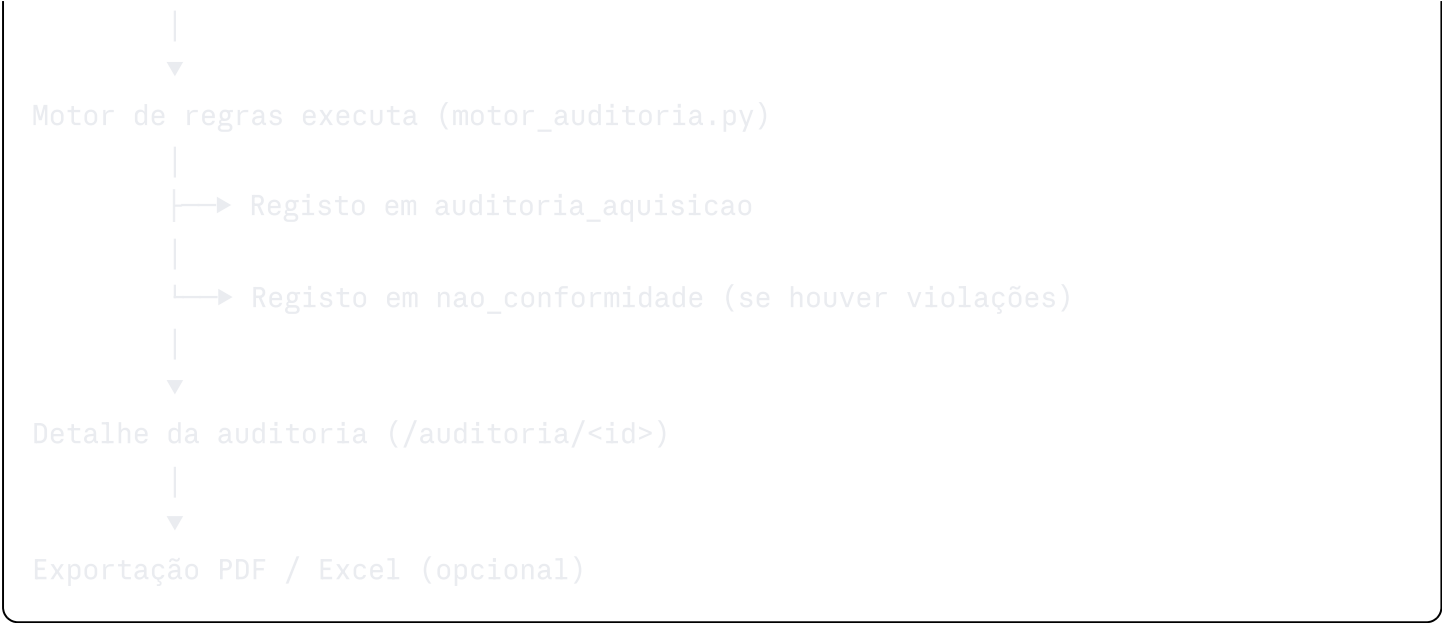


Estrutura de Blueprints

Blueprint	Prefixo URL	Responsabilidade
main	/	Dashboard e página inicial
auth_bp	/auth	Autenticação — login e logout
auditoria_bp	/auditoria	Gestão de sessões de auditoria
aquisicao_bp	/aquisicoes	Listagem e selecção de aquisições
nc_bp	/nao-conformidades	Gestão de não conformidades
config_bp	/configuracao	Administração do sistema

Fluxo principal da aplicação





3. Base de Dados

Modelo de dados — 10 tabelas

utilizador

Utilizadores internos do sistema de auditoria. Separado de `colaborador` porque são entidades com papéis distintos — o utilizador acede ao sistema, o colaborador é referenciado nos processos de aquisição.

Campo	Tipo	Descrição
id_utilizador	INT PK	Identificador único
nome	VARCHAR(100)	Nome completo
email	VARCHAR(150) UNIQUE	Email de acesso
password_hash	VARCHAR(255)	Hash da password (scrypt)
perfil	ENUM	<code>auditor</code> ou <code>administrador</code>
ativo	BOOLEAN	Estado da conta

colaborador

Importado do ERP via CSV. Representa quem solicita e aprova aquisições na organização.

Campo	Tipo	Descrição
id_colaborador	INT PK	Identificador único
nome	VARCHAR(100)	Nome completo
email	VARCHAR(150) UNIQUE	Email institucional
nivel_hierarquico	INT (1-6)	Nível na hierarquia organizacional
ativo	BOOLEAN	Se o colaborador está activo

Escala hierárquica:

- 1 = Técnico
- 2 = Supervisor
- 3 = Chefe de Departamento
- 4 = Subdirector
- 5 = Director
- 6 = Administrador

centro_custo

Unidade contabilística que agrupa despesas por departamento.

Campo	Tipo	Descrição
id_centro	INT PK	Identificador único
nome	VARCHAR(100)	Nome do centro
departamento	VARCHAR(100)	Departamento associado
id_responsavel	INT FK → colaborador	Colaborador responsável
ativo	BOOLEAN	Estado do centro

orcamento

Envelope orçamental anual por centro de custo.

Campo	Tipo	Descrição
id_orcamento	INT PK	Identificador único
id_centro	INT FK → centro_custo	Centro de custo associado
periodo	VARCHAR(20)	Período orçamental (ex: 2025)
valor_orcado	DECIMAL(15,2)	Valor aprovado
valor_executado	DECIMAL(15,2)	Valor já executado (do ERP)

Constraint: UNIQUE (id_centro, periodo) — um orçamento por centro por período.

aquisicao

Evento auditável central. Importado do ERP via CSV.

Campo	Tipo	Descrição
id_aquisicao	INT PK	Identificador único
id_centro	INT FK → centro_custo	Centro de custo
id_orcamento	INT FK → orcamento	Orçamento associado
id_solicitante	INT FK → colaborador	Quem solicitou
id_aprovador	INT FK → colaborador	Quem aprovou
data_solicitacao	DATE	Data da solicitação
data_aprovacao	DATE NULL	Data da aprovação
valor	DECIMAL(15,2)	Valor da aquisição
descricao	VARCHAR(255)	Objecto da aquisição
tipo_aquisicao	ENUM	bem ou servico
documento_referencia	VARCHAR(100) NULL	Referência documental
status_aprovacao	ENUM	aprovado, pendente, rejeitado
origem_dado	ENUM	csv, erp, manual
confirmacao_recepcao	BOOLEAN	Se o bem/serviço foi recebido

Nota arquitectural: `id_solicitante` e `id_aprovador` são dois FKs distintos para a mesma tabela `colaborador`. Esta duplicação intencional suporta directamente RN07 — o motor verifica `id_solicitante == id_aprovador`.

`regra_auditoria`

Catálogo de regras de negócio configuráveis pelo administrador.

Campo	Tipo	Descrição
id_regra	INT PK	Identificador único
codigo	VARCHAR(10) UNIQUE	Código formal (ex: RN01)
nome	VARCHAR(100)	Nome descritivo
descricao	TEXT	Descrição completa da regra
campo	VARCHAR(80)	Atributo da aquisição avaliado
operador	ENUM	Tipo de comparação
valor_referencia	DECIMAL(15,2) NULL	Limiar configurável
tipo_regra	ENUM	<code>orcamental</code> , <code>autorizacao</code> , <code>procedimental</code> , <code>integridade</code>
gravidade	ENUM	<code>baixa</code> , <code>media</code> , <code>alta</code> , <code>critica</code>
ativa	BOOLEAN	Se a regra está activa

Operadores disponíveis: `igual`, `diferente`, `maior`, `maior_igual`, `menor`, `menor_igual`, `nulo`, `nao_nulo`, `igual_campos`

`limiar_autorizacao`

Extensão selectiva de `regra_auditoria` para RN05 e RN06. Define os níveis hierárquicos mínimos exigidos por faixa de valor.

Campo	Tipo	Descrição
id_limiar	INT PK	Identificador único
id_regra	INT FK → regra_auditoria	Regra associada
valor_minimo	DECIMAL(15,2)	Limite inferior da faixa
valor_maximo	DECIMAL(15,2) NULL	Limite superior (NULL = sem tecto)
nivel_minimo	INT (1-6)	Nível hierárquico mínimo exigido

auditoria

Registo de cada sessão de auditoria executada.

Campo	Tipo	Descrição
id_auditoria	INT PK	Identificador único
id_utilizador	INT FK → utilizador	Auditor que executou
data_execucao	DATETIME	Data e hora de execução
periodo_analisado	VARCHAR(20)	Período auditado
total_transacoes	INT	Total de aquisições analisadas
total_nao_conformidades	INT	Total de não conformidades
status	ENUM	em_curso, concluida, erro

auditoria_aquisicao

Tabela de junção com semântica própria entre **auditoria** e **aquisicao**. Regista o resultado da avaliação de cada aquisição numa sessão.

Campo	Tipo	Descrição
id_auditoria_aquisicao	INT PK	Identificador único
id_auditoria	INT FK → auditoria	Sessão de auditoria
id_aquisicao	INT FK → aquisicao	Aquisição avaliada
resultado	ENUM	conforme, nao_conforme, inconclusivo

Constraint: UNIQUE (id_auditoria, id_aquisicao) — cada aquisição é avaliada uma única vez por sessão.

nao_conformidade

Registo de cada violação detectada pelo motor.

Campo	Tipo	Descrição
id_nao_conformidade	INT PK	Identificador único
id_auditoria_aquisicao	INT FK → auditoria_aquisicao	Avaliação que gerou a violação
id_regra	INT FK → regra_auditoria	Regra violada
descricao	TEXT	Descrição da irregularidade
gravidade	ENUM	Gravidade herdada da regra
data_registo	DATETIME	Data e hora de detecção
status	ENUM	aberta, em_analise, resolvida, ignorada
comentario_auditor	TEXT NULL	Observações do auditor

Índices de performance

```
sql

-- aquisicao
idx_aquisicao_centro, idx_aquisicao_orcamento
idx_aquisicao_solicitante, idx_aquisicao_aprovador
idx_aquisicao_data

-- nao_conformidade
idx_nc_gravidade, idx_nc_status
idx_nc_regra, idx_nc_data

-- auditoria
idx_auditoria_periodo, idx_auditoria_status

-- auditoria_aquisicao
idx_aa_resultado
```

4. Motor de Regras

O motor de regras é o componente central e academicamente diferenciador do sistema. Vive em `app/services/motor_auditoria.py`.

Princípio de funcionamento

O motor é orientado a dados — as regras não estão hardcoded no código mas sim na tabela `regra_auditoria`. Isto significa que:

- Adicionar uma nova regra não requer alteração de código
- Desactivar uma regra não apaga o histórico de violações anteriores
- Os limiares numéricos são configuráveis pelo administrador em runtime

As 16 regras implementadas

Código	Nome	Tipo	Gravidade
RN01	Saldo orçamental insuficiente	Orçamental	Crítica
RN02	Centro de custo inválido	Orçamental	Crítica
RN03	Período orçamental não definido	Orçamental	Alta
RN04	Dados obrigatórios em falta	Orçamental	Crítica
RN05	Aprovação hierárquica insuficiente	Autorização	Crítica
RN06	Perfil de aprovador incompatível	Autorização	Alta
RN07	Solicitante igual ao aprovador	Autorização	Crítica
RN08	Solicitante não identificado	Procedimental	Alta
RN09	Documentação em falta	Procedimental	Alta
RN10	Confirmação de recepção em falta	Procedimental	Alta
RN11	Identificação única em falta	Integridade	Crítica
RN12	Inconsistência de datas	Integridade	Alta
RN13	Registo duplicado	Integridade	Crítica
RN14	Classificação automática	Integridade	Crítica
RN15	Registo de não conformidade	Integridade	Alta
RN16	Relatório de auditoria	Integridade	Média

Lógica de execução

python

```
def executar_auditoria(auditoria, ids_aquisicao):
    # 1. Carrega regras activas da base de dados
    regras = RegraAuditoria.query.filter_by(ativa=True).all()

    # 2. Carrega aquisições seleccionadas ordenadas cronologicamente
    aquisicoes = Aquisicao.query.filter(
        Aquisicao.id_aquisicao.in_(ids_aquisicao)
    ).order_by(Aquisicao.data_aprovacao.asc()).all()

    for aquisicao in aquisicoes:
        # PASSO 1 – Avalia todas as regras
        violacoes = []
        for regra in regras:
            houve_violacao, descricao = avaliar_regra(aquisicao, regra, regra_rn05)
            if houve_violacao:
                violacoes.append((regra, descricao))

        # PASSO 2 – Regista resultado em auditoria_aquisicao
        resultado = 'nao_conforme' if violacoes else 'conforme'
        aa = AuditoriaAquisicao(...)
        db.session.flush() # Obtém o id gerado

        # PASSO 3 – Regista não conformidades
        for regra, descricao in violacoes:
            registrar_nao_conformidade(aa.id_auditoria_aquisicao, regra, descricao)

    db.session.commit()
```

Lógica especial de RN01 — Saldo sequencial

RN01 é a regra mais complexa. O saldo disponível não é fixo — depende de quando cada aquisição foi aprovada. O motor usa lógica sequencial:

```
Saldo disponível para aquisição X =
    valor_orcado
    - valor_executado (histórico do ERP)
    - soma das aquisições aprovadas ANTES de X no mesmo orçamento
```

As aquisições são ordenadas por `data_aprovacao` ascendente — garantindo que o cálculo reflecte o saldo real no momento de cada aprovação.

Lógica especial de RN05/RN06 — Limiares hierárquicos

RN05 e RN06 são avaliadas em conjunto. O motor consulta a tabela `limiar_autorizacao` para determinar o nível mínimo exigido para o valor da aquisição:

```
python

limiar = buscar_limiar(regra_rn05.id_regra, aquisicao.valor)
if limiar:
    aprovador = Colaborador.query.get(aquisicao.id_aprovador)
    if aprovador.nivel_hierarquico < limiar.nivel_minimo:
        # Violação de RN05 e RN06
```

Templates de descrição das violações

Cada regra tem um template de descrição que gera mensagens dinâmicas com os dados reais:

```
python

DESCRICOES_VIOLACAO = {
    'RN01': lambda a, saldo: (
        f"Saldo insuficiente – valor ({a.valor} Kz) excede "
        f"saldo disponível ({saldo} Kz) no centro {a.id_centro}."
    ),
    'RN07': lambda a, **_: (
        f"Solicitante igual ao aprovador – colaborador "
        f"id {a.id_solicitante} nos dois campos."
    ),
    ...
}
```

5. Módulos da Aplicação

`app/routes/main.py` — Dashboard

Calcula e serve os indicadores do dashboard:

- Totais de auditorias, conformes, não conformes e críticas
- Dados para gráfico de barras por centro de custo
- Dados para gráfico de rosca por gravidade
- Últimas auditorias e não conformidades

`app/routes/auditoria.py` — Gestão de Auditorias

Rotas:

- `GET /auditoria` — lista todas as sessões
- `GET /auditoria/iniciar` — redireciona para aquisições com mensagem
- `POST /auditoria/nova` — confirmação e execução da auditoria
- `GET /auditoria/<id>` — detalhe de uma sessão
- `GET /auditoria/<id>/exportar/pdf` — exporta em PDF
- `GET /auditoria/<id>/exportar/excel` — exporta em Excel

Fluxo de nova auditoria:

1. Utilizador selecciona aquisições em `/aquisicoes`
2. Submete formulário com IDs seleccionados
3. `/auditoria/nova` mostra confirmação com lista de aquisições
4. Utilizador confirma — motor executa
5. Redireciona para detalhe da auditoria

`app/routes/aquisicao.py` — Listagem de Aquisições

Filtros disponíveis:

- Por centro de custo
- Por período orçamental
- Por status de aprovação

Permite selecção múltipla via checkboxes para envio à auditoria.

`app/routes/nao_conformidade.py` — Gestão de Não Conformidades

Filtros disponíveis:

- Por gravidade
- Por status
- Por regra violada

Permite actualização de status e adição de comentários via modal.

`app/routes/configuracao.py` — Administração

Hub com três módulos:

Gestão de Utilizadores (apenas administradores):

- Criar utilizador com perfil e password
- Activar/desactivar utilizador
- Protecção: administrador não pode desactivar a própria conta

Gestão de Regras (administradores editam, auditores vêem):

- Activar/desactivar regras
- Editar `valor_referencia`
- Editar limiares de autorização

Importação de Dados (administradores e auditores):

- Upload de CSV por entidade
- Validação linha a linha
- Relatório de erros por linha

`app/routes/auth.py` — Autenticação

- `GET/POST /auth/login` — formulário de login
- `GET /auth/logout` — terminar sessão

Validações no login:

- Email deve existir na base de dados
 - Utilizador deve estar activo (`ativo=True`)
 - Password verificada contra hash com `check_password_hash`
 - Redirecionamento para URL original após login (`next` parameter)
-

6. Segurança e Autenticação

Gestão de passwords

As passwords são armazenadas como hash usando o algoritmo **scrypt** via `werkzeug.security`:

```
python

# Definir password
user.password_hash = generate_password_hash('password')

# Verificar password
check_password_hash(user.password_hash, 'password')
```

O scrypt é um algoritmo de derivação de chave resistente a ataques de força bruta por ser intensivo em memória.

Protecção de rotas

Todas as rotas (excepto login) estão protegidas com `@login_required` do Flask-Login. Tentativas

de acesso sem autenticação redirecionam para `/auth/login` com o URL original preservado no parâmetro `next`.

Controlo de acesso por perfil

Rotas administrativas verificam `current_user.is_admin`:

```
python

def admin_required():
    if not current_user.is_admin:
        flash('Acesso restrito a administradores.', 'danger')
        return False
    return True
```

Separação Utilizador / Colaborador

Esta é uma decisão arquitectural deliberada. O `Utilizador` é a entidade do sistema de auditoria — quem acede à plataforma. O `Colaborador` é importado do ERP — quem solicita e aprova aquisições. Um auditor do sistema pode ser uma pessoa completamente diferente dos colaboradores da organização auditada.

Limitações de segurança conhecidas (MVP)

- **CSRF:** Não implementado nesta versão. Requer Flask-WTF em versão de produção.
- **Rate limiting:** Login não tem limite de tentativas. Requer Flask-Limiter.
- **Headers HTTP:** Sem `X-Frame-Options`, `Content-Security-Policy`. Requer Flask-Talisman.

7. Importação de Dados

Serviço `app/services/importacao.py`

O módulo de importação suporta quatro entidades com validação linha a linha.

Ordem obrigatória de importação

```
1. colaboradores.csv      ← sem dependências
2. centros_custo.csv      ← depende de colaboradores
3. orcamentos.csv         ← depende de centros de custo
4. aquisicoes.csv         ← depende de tudo
```

Estrutura dos CSVs

`colaboradores.csv`

centros_custo.csv

```
id_centro,nome,departamento,responsavel_id,ativo
1,CC Tecnologia,Tecnologia e Sistemas,2,1
```

orcamentos.csv

```
id_orcamento,id_centro,periodo,valor_orcado,valor_executado
1,1,2025,45000000.00,28500000.00
```

aquisicoes.csv

```
id_aquisicao,data_solicitacao,data_aprovacao,valor,descricao,
tipo_aquisicao,documento_referencia,status_aprovacao,origem_dado,
confirmacao_recepcao,id_solicitante,id_aprovador,id_centro,id_orcamento
```

Validações aplicadas

Para cada linha de cada CSV:

- Campos obrigatórios presentes e não vazios
- Tipos de dados correctos (inteiros, decimais, datas, booleanos)
- Referências a entidades existentes na base de dados
- Valores dentro dos intervalos válidos (ex: nível hierárquico entre 1 e 6)

Linhas inválidas são rejeitadas individualmente — a importação não para por causa de um erro numa linha.

Lógica de upsert

Para colaboradores, centros de custo e orçamentos: se o registo já existe, é actualizado. Para aquisições: cada linha cria sempre um novo registo — uma aquisição é um evento único e irrepetível.

8. Exportação de Relatórios

Serviço `app/services/exportacao.py`

Gera relatórios em PDF e Excel a partir de uma sessão de auditoria.

Relatório PDF (ReportLab)

Estrutura:

1. Cabeçalho com identidade visual da NS Aplicação
2. Dados da sessão — período, auditor, data, status
3. Tabela de resultados por aquisição — conformes a preto, não conformes a vermelho
4. Tabela de não conformidades com descrição completa, regra, gravidade e status
5. Rodapé com data de geração

Implementação: gerado em memória (`BytesIO`) sem escrita em disco. Devolvido ao browser via `send_file` com `as_attachment=True`.

Relatório Excel (openpyxl)

Três folhas:

- **Resumo** — dados gerais da sessão com formatação condicional
- **Resultados por Aquisição** — uma linha por aquisição com resultado colorido
- **Não Conformidades** — detalhe completo com cores por gravidade

Formatação: células com cores por resultado (verde/vermelho), larguras de coluna ajustadas, formatação numérica para valores monetários.

9. Interface Web

Layout

Sidebar fixa à esquerda com toggle de recolher/expandir. Transições CSS suaves. Conteúdo principal ocupa o espaço restante com margem ajustável.

Componentes principais

Dashboard:

- 4 cards de indicadores (auditorias, conformes, não conformes, críticas)
- Gráfico de barras — conformes vs não conformes por centro de custo (Chart.js)
- Gráfico de rosca — distribuição por gravidade (Chart.js)
- Tabela das últimas auditorias
- Tabela das últimas não conformidades

Aquisições:

- Filtros por centro de custo, período e status

- Tabela com checkboxes de selecção múltipla
- Botão "Auditar seleccionadas" activado dinamicamente por JavaScript
- Contador de seleccionadas actualizado em tempo real

Nova Auditoria:

- Formulário de confirmação com lista de aquisições a auditar
- Auditor identificado automaticamente pelo utilizador autenticado

Detalhe da Auditoria:

- Cards de resumo — período, total, não conformes, status
- Tabela de resultados por aquisição
- Não conformidades expandidas por linha
- Botões de exportação PDF e Excel

Não Conformidades:

- Filtros por gravidade, status e regra
- Modal de actualização por linha — status e comentário do auditor

Configuração — Hub:

- Card de Gestão de Utilizadores (só administradores)
- Card de Gestão de Regras (todos)
- Card de Importação de Dados (todos)

Feedback visual por perfil

O badge do utilizador na sidebar muda de cor por perfil:

- Administrador: vermelho
- Auditor: azul

Na página de configuração, o card de Gestão de Utilizadores só aparece para administradores. Na página de regras, os botões de edição e toggle só aparecem para administradores.

10. Estrutura de Ficheiros

```
ns_auditoria/
|
├─ run.py           # Ponto de entrada da aplicação
├─ .env             # Variáveis de ambiente (não versionado)
├─ requirements.txt # Dependências Python
```

```
└─ .gitignore                                # Ficheiros excluídos do Git
|
└─ app/
|   └─ __init__.py                          # Application Factory + registo de blueprints
|   └─ config.py                            # Configurações da aplicação
|   |
|   └─ models/                              # Modelos SQLAlchemy (camada de dados)
|       └─ __init__.py
|       └─ utilizador.py
|       └─ colaborador.py
|       └─ centro_custo.py
|       └─ orcamento.py
|       └─ aquisicao.py
|       └─ regra_auditoria.py
|       └─ limiar_autorizacao.py
|       └─ auditoria.py
|       └─ auditoria_aquisicao.py
|       └─ nao_conformidade.py
|   |
|   └─ routes/                              # Blueprints (camada de apresentação)
|       └─ main.py
|       └─ auth.py
|       └─ auditoria.py
|       └─ aquisicao.py
|       └─ nao_conformidade.py
|       └─ configuracao.py
|   |
|   └─ services/                            # Lógica de negócio (camada de negócio)
|       └─ __init__.py
|       └─ motor_auditoria.py
|       └─ importacao.py
|       └─ exportacao.py
|   |
|   └─ templates/                           # Templates Jinja2
|       └─ base.html
|       └─ index.html
|       └─ auth/
|           └─ login.html
|       └─ auditoria/
|           └─ index.html
|           └─ nova.html
|           └─ detalhe.html
|       └─ aquisicao/
|           └─ index.html
|       └─ nao_conformidade/
|           └─ index.html
|       └─ configuracao/
```

11. Decisões Técnicas

Porquê Flask e não Django?

Flask é um microframework que oferece controlo granular sobre a arquitectura. Django impõe convenções (ORM, admin, formulários) que criariam fricção com a arquitectura de motor de regras parametrizável. Flask permite construir exactamente o que o sistema precisa sem overhead desnecessário.

Porquê MySQL e não PostgreSQL?

MySQL estava disponível no ambiente de desenvolvimento sem configuração adicional. Para o volume de dados do MVP, não há diferença funcional relevante. O sistema usa apenas funcionalidades SQL standard — a migração para PostgreSQL seria trivial.

Porquê separar Utilizador de Colaborador?

O sistema não é transaccional — não aprova nem rejeita aquisições. O auditor (Utilizador) é quem usa a plataforma. O colaborador é quem participou nas aquisições auditadas. São entidades com

ciclos de vida distintos — um auditor pode auditar dados de colaboradores que já saíram da organização.

Porquê as regras estão na base de dados e não no código?

Esta é a decisão arquitectural mais importante. Manter as regras na base de dados transforma o motor num Business Rules Engine — adicionar ou modificar uma regra não requer alteração de código nem deployment. O administrador ajusta os parâmetros pela interface. Esta abordagem é documentada por Von Halle (2001) e Ross (2013).

Porquê lógica sequencial em RN01?

A alternativa seria verificar se o total de aquisições do período cabe no orçamento — mas isso não reflecte a realidade. O auditor precisa de saber em que momento o saldo se esgotou, não apenas se o total excede o orçamento. A lógica sequencial por `data_aprovacao` responde exactamente a essa questão.

Porquê não implementar CSRF nesta versão?

O sistema é um MVP académico desenvolvido em ambiente controlado com utilizadores conhecidos. A implementação de CSRF com Flask-WTF é uma melhoria de segurança documentada para versões futuras — não uma falha de design mas uma decisão consciente de scope do MVP.

12. Limitações e Trabalho Futuro

Limitações documentadas do MVP

Limitação	Impacto	Versão Futura
Sem protecção CSRF	Segurança — vulnerável a ataques externos	Flask-WTF
Sem rate limiting no login	Segurança — força bruta possível	Flask-Limiter
Sem headers de segurança HTTP	Segurança — XSS e clickjacking	Flask-Talisman
RN02, RN03, RN04, RN08 garantidas pelo ERP	Cobertura — não detectadas pelo motor	Módulo de staging
<code>valor_executado</code> não actualizado dinamicamente	Precisão do saldo	Recálculo automático
Sem paginação nas listagens	Performance com grandes volumes	Flask-SQLAlchemy pagination

Limitação	Impacto	Versão Futura
Query N+1 no dashboard	Performance	Query com GROUP BY
Sem log de acções dos utilizadores	Auditabilidade do sistema	Tabela de audit log

Funcionalidades para versões futuras

- 1. **Autenticação multi-factor** — 2FA para administradores
- 2. **Log de acções** — registo de quem alterou o quê e quando
- 3. **API REST** — integração directa com ERPs sem CSV
- 4. **Notificações** — alertas por email quando não conformidades críticas são detectadas
- 5. **Dashboard analítico avançado** — tendências temporais e comparação entre períodos
- 6. **Módulo de staging** — tabela intermédia para dados com erros de referência
- 7. **Gestão de perfis granular** — permissões por módulo em vez de apenas por perfil